
Gestures



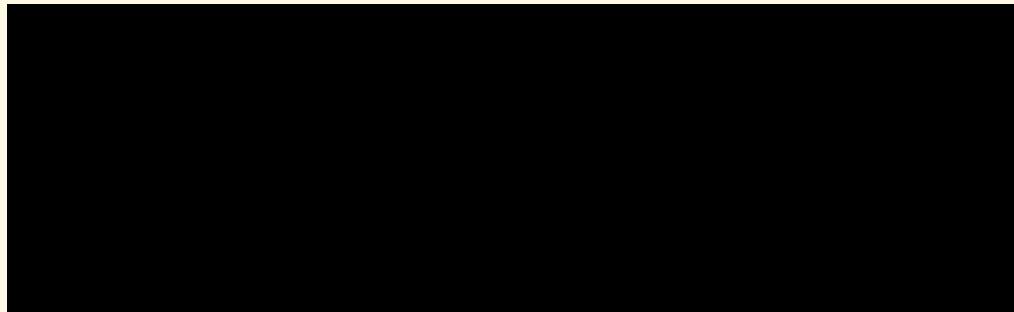
Interaction

- ▶ Basic interaction (already visited)
 - ▶ Tapping with `onPress` in `<Button>`
 - ▶ Touching in `<Touchable*>` components
 - ▶ Scrolling in `<ScrollView>`, `<FlatList>` components
- ▶ Other gestures
 - ▶ Swiping, sliding
 - ▶ Multi-touch gestures
 - ▶ Custom gestures

Gesture Responder System

Gesture Responder System

- ▶ Basic system that manages the lifecycle of gestures in the app
- ▶ Allows components to negotiate the touch interactions
 - ▶ A touch can go through several phases when determining the user's intention
- ▶ Responder lifecycle: Start → Move → Release



Responder Negotiation

- ▶ Two method props to ask the view if the view wants to become responder:

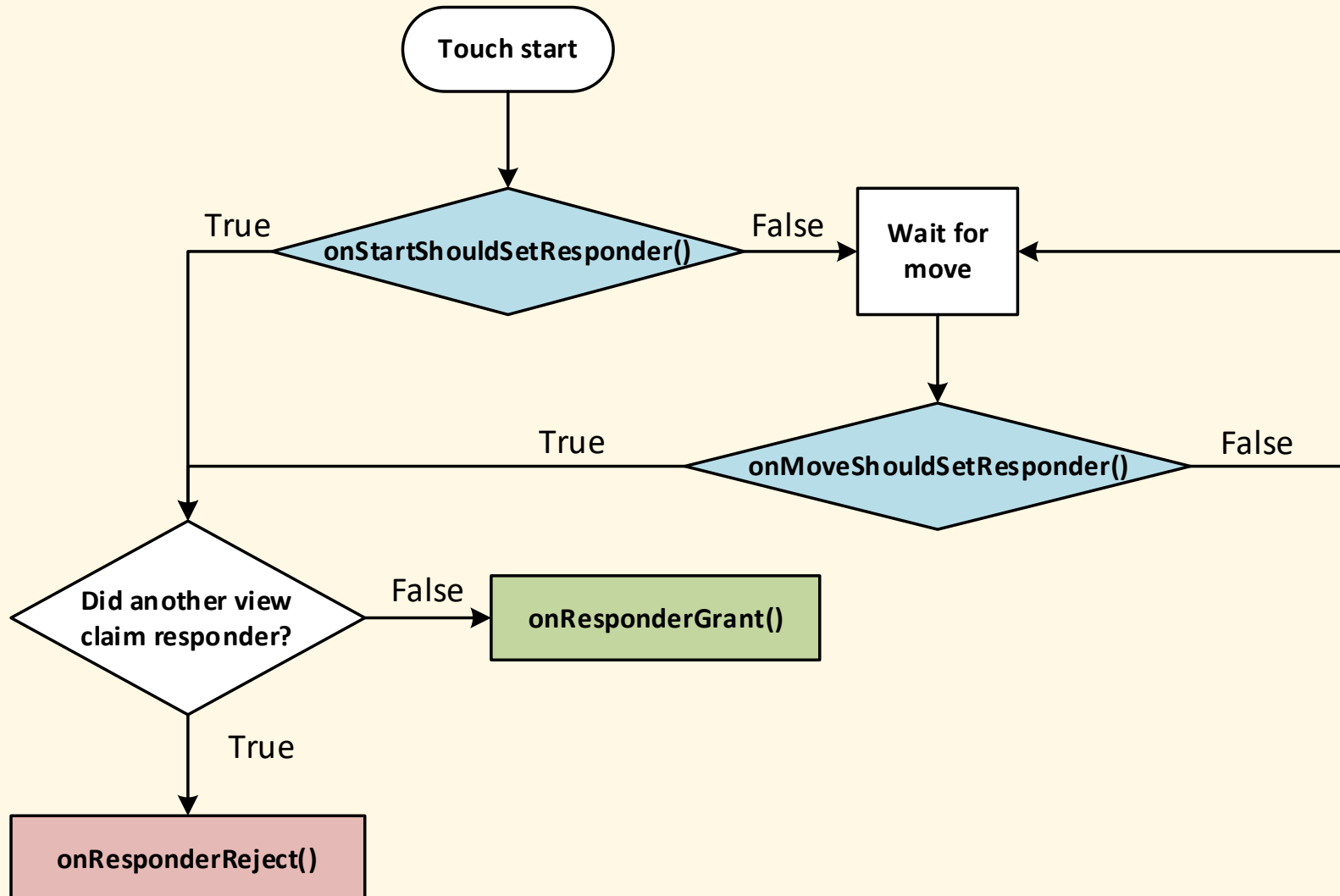


- ▶ `onStartShouldSetResponder: (evt) => Boolean`
Does this view want to become responder on the start of a touch?
- ▶ `onMoveShouldSetResponder: (evt) => Boolean`
Does this view want to “claim” touch responsiveness?
Called for every touch move on the view when it is not the responder.
- ▶ Attn: Even when these methods return true, the view may not be granted if another view is being the responder and does not want to release.
➔ See `onResponderTerminationRequest` prop.

Responder Grant & Reject Notification

- ▶ Two method props allowing to know when a view is granted/rejected to become responder:
 - ▶ `onResponderGrant: (evt) => {}`
The view is now responding for touch events.
This is the time to highlight and show the user what is happening.
 - ▶ `onResponderReject: (evt) => {}`
Something else is the responder right now and will not release it.

Responder Negotiation Summary



Response Handlers

- ▶ Method props called when the view is responding:
 - ▶ `onResponderMove: (evt) => {}`
The user is moving their finger.
 - ▶ `onResponderRelease: (evt) => {}`
Fired at the end of the touch, i.e. “touch up”.
 - ▶ `onResponderTerminationRequest: (evt) => Boolean`
Another view asks to become responder, should this view release the responder?
 - ▶ `onResponderTerminate: (evt) => {}`
The responder has been taken from the view. Might be taken by other views after `onResponderTerminationRequest`, or by the OS without asking.

Event Structure

▶ nativeEvent

- ▶ `changedTouches`: Array of all touch events that have changed since the last event.
- ▶ `identifier`: The ID of the touch.
- ▶ `locationX`: The X position of the touch, relative to the element.
- ▶ `locationY`: The Y position of the touch, relative to the element.
- ▶ `pageX`: The X position of the touch, relative to the root element.
- ▶ `pageY`: The Y position of the touch, relative to the root element.
- ▶ `target`: The node ID of the element receiving the touch event.
- ▶ `timestamp`: A time identifier for the touch, useful for velocity calculation.
- ▶ `touches`: Array of all current touches on the screen.

Usage Pattern

```
function ExampleComponent() {  
  return <View  
    onStartShouldSetResponder={() => true}  
    onMoveShouldSetResponder={() => true}  
    onResponderGrant={({nativeEvent}) => {...}}  
    onResponderMove={({nativeEvent}) => {...}}  
    onResponderRelease={({nativeEvent}) => {...}}  
    ...  
  />  
}
```

Exercise

- ▶ Create a component responding to touch states (Start, Move, Release) with corresponding background colors.

Capture *ShouldSetResponder Handlers

- ▶ *ShouldSetResponder handlers are called with bubbling pattern: deepest node is called first and becomes responder when multiple views return true.
- ▶ If a parent wants to become responder: using the **capture phase**, which happens before the responder system bubbles up
 - ▶ `onStartShouldSetResponderCapture: (evt) => Boolean`
 - ▶ `onMoveShouldSetResponderCapture: (evt) => Boolean`

Exercises

1. Create two nested components, both responding to touch states (Start, Move, Release) with corresponding background colors.
2. Modify the app to make the parent component to capture touches on the child.

PanResponder

Introduction

- ▶ Gesture Responder System is generic, but hard to use in simple use cases
- ▶ `PanResponder` is built on top of Gesture Responder System to facilitate:
 - ▶ Providing a `gestureState` which includes
 - ▶ `stateID`: ID of the `gestureState` - persisted as long as there's at least one touch on screen.
 - ▶ `moveX`, `moveY`: The latest screen coordinates of the recently-moved touch.
 - ▶ `x0`, `y0`: The screen coordinates of the responder grant.
 - ▶ `dx`, `dy`: Accumulated distance of the gesture since the touch started.
 - ▶ `vx`, `vy`: Current velocity of the gesture.
 - ▶ `numberActiveTouches`: Number of touches currently on screen.
 - ▶ Holding an `InteractionManager` handle to block long-running JS events from interrupting active gestures

Method Props

- ▶ `onStartShouldSetPanResponder: (evt, gestureState) => Boolean`
- ▶ `onStartShouldSetPanResponderCapture: (evt, gestureState) => Boolean`
- ▶ `onMoveShouldSetPanResponder: (evt, gestureState) => Boolean`
- ▶ `onMoveShouldSetPanResponderCapture: (evt, gestureState) => Boolean`
- ▶ `onPanResponderGrant: (evt, gestureState) => {}`
- ▶ `onPanResponderMove: (evt, gestureState) => {}`
- ▶ `onPanResponderTerminationRequest: (evt, gestureState) => Boolean`
- ▶ `onPanResponderRelease: (evt, gestureState) => {}`
- ▶ `onPanResponderTerminate: (evt, gestureState) => {}`
- ▶ `onShouldBlockNativeResponder: (evt, gestureState) => {}`

Returns whether this component should block native components from becoming the JS responder. Returns true by default.

Usage Pattern

```
function ExampleComponent() {
  const panResponder = React.useRef(
    PanResponder.create({
      onStartShouldSetPanResponder: () => true,
      onStartShouldSetPanResponderCapture: ...,
      onMoveShouldSetPanResponder: () => true,
      onMoveShouldSetPanResponderCapture: ...,
      onPanResponderGrant: ...,
      onPanResponderMove: ...,
      onPanResponderTerminationRequest: ...,
      onPanResponderRelease: ...,
      onPanResponderTerminate: ...,
      onShouldBlockNativeResponder: ...
    })
  ).current;

  return <View {...panResponder.panHandlers} />;
}
```

Example: Draggable Box

```
function Box() {
  const [offLast, setOffsetLast] = useState({dx: 0, dy: 0});
  const [offDrag, setOffsetDrag] = useState({dx: 0, dy: 0});

  const panResponder = useMemo(() => PanResponder.create({
    onMoveShouldSetPanResponder: () => true,
    onPanResponderGrant: () => setOffsetDrag({dx: 0, dy: 0 }),
    onPanResponderMove: (evt, gs) => setOffsetDrag({dx: gs.dx, dy: gs.dy}),
    onPanResponderRelease: (evt, gs) => {
      setOffsetLast({dx: offLast.dx + gs.dx, dy: offLast.dy + gs.dy});
      setOffsetDrag({dx: 0, dy: 0});
    }
  })), [offLast]);

  return <View style={[ styles.box, { transform: [
    { translateX: offLast.dx + offDrag.dx },
    { translateY: offLast.dy + offDrag.dy } ]}] }
    {...panResponder.panHandlers} />
}
```

- ▶ **Attn:** useMemo instead of useRef allows panResponder listeners have up-to-date value of offLast

Exercises

1. Make the draggable box bounded by its parent view.
2. Determine if a gesture is swipe to left, swipe to right.

Homework

- ▶ Make a list whose items response to left/right swipe gestures:
 - ▶ Remove the item when user swipes it to right.
 - ▶ Highlight/unhighlight the item when user swipes it to left.

